NeoCruncher and the Kolmogorov Engine:
An open-source hybrid-based computation project

Markov–Nekrasov Research Team

18 October 2026

Release Paper | Neocruncher 1.0 (Beta)

Abstract

High-precision constant computation combines rapidly convergent mathematical identities with integer arithmetic, scheduling, memory management, and radix conversion. NeoCruncher is a free and open-source command-line implementation that exposes CPU, OpenCL GPU, and hybrid CPU/GPU execution paths through a common workflow. This paper describes its architecture and the Kolmogorov arithmetic infrastructure, including PyKolm, an experimental CPU planning and subtree-execution layer. We derive the binary-splitting recurrence used for Chudnovsky's formula, explain integer interval finalization, and distinguish numerical correctness arguments from independent output comparisons. A separate sequential auditor preserves timings, resource samples, source identities, and unsuccessful trials. A supplied CPU audit reports completion of one billion decimal places of pi in 1174.76 seconds of calculation time and 1176.48 seconds of process wall time, with a sampled process-tree RSS peak of 15.68 GiB. Independent comparison in that audit covers the first 100,000 places. A newer hybrid console report records 100 million places in 1929.621 seconds; its complete metadata remains unavailable for review. At matching sizes, the reported hybrid wall times exceed CPU wall times. The contribution is an inspectable implementation and a documented evaluation workflow, with explicit limitations; no new constant formula, GPU speedup, or performance record is claimed.

1 Introduction

Computing a mathematical constant to many decimal places is not simply a matter of evaluating a familiar formula with a larger floating-point type. The choice of series determines how many terms are necessary. The organization of the sum determines the sizes and number of integer operations. The arithmetic library, parallel schedule, and memory representation determine their cost. Finally, the program must establish which digits are justified and convert an internal binary representation into decimal output. A fast multiplication kernel alone does not resolve this complete problem.

NeoCruncher provides a terminal application for exploring these layers within one open-source project. Its numerical backends include a CPU implementation, a GPU-resident OpenCL path, and a host-assisted hybrid path. Kolmogorov supplies arithmetic infrastructure and integration interfaces; PyKolm explores explicit CPU execution plans. These names identify software components, not new mathematical identities or complexity results.

The project's initial delivery scope is computation at 100 million decimal places on CPU and 10 million in hybrid mode on the tested configurations. The subsequently reported CPU billion-place and hybrid hundred-million-place runs extend the observed completion range. They do not expand the independently checked prefix or establish portable hardware guarantees. GPU-only execution remains experimental at large sizes.

The scope of this paper is: (i) to provide an architectural account that states where arithmetic occurs in each backend; (ii) an explanation of the exact recurrences and interval-based

digit extraction used by the implementation; (iii) a definition of PyKolm's execution-plan and normalization semantics; and (iv) an evaluation protocol that separates completion, integrity, numerical comparison, and resource measurement. Detailed numerical tables, phase breakdowns, and provenance are provided in the companion *NeoCruncher Alpha Release 1.0: Benchmark Report*.

This is a software and methods paper. Its measured results are preliminary, with one supplied observation per reported size. It neither proposes a new formula for pi nor establishes that heterogeneous execution is faster than an optimized CPU implementation. The present software remains subject to asset review and final author approval.

2 Background and related work

2.1 Exact arithmetic and evaluation of rational series

Multiple-precision arithmetic represents an integer using a sequence of machine words, or limbs. The number of limbs grows with the value. Exact integer operations preserve their mathematical result subject to implementation correctness and available resources. This differs from fixed-precision floating-point operations, which generally round intermediate values.

Binary splitting is a well-established organization for evaluating rational series. Haible and Papanikolaou describe a general framework in which products and partial sums are accumulated over intervals [1]. Brent and Zimmermann provide the broader arithmetic context, including multiplication, division, square roots, and conversion algorithms [2]. NeoCruncher uses these established ideas; their inclusion here is explanatory rather than a claim of invention.

The CPU implementation benefits from GMP through gmpy2 when available. GMP selects among multiplication algorithms according to operand sizes and build configuration, including basecase, Karatsuba, Toom variants, and FFT-based methods [3]. Calling this backend a "Python CPU implementation" does not mean its large multiplications are Python loops: the coordinating logic is Python, while GMP executes substantial arithmetic in native code.

2.2 Alexander J. Yee and y-cruncher

Alexander J. Yee's work on y-cruncher is an important technical reference and source of inspiration for NeoCruncher. His published account of binary splitting explains reusable recursion families [4]; his formulas page discusses computation and verification choices [5]. We explicitly credit this work in both the technical background and the acknowledgements. Binary splitting and the underlying constant identities also have their own earlier literature and authors.

Yee's historical *Hybrid NTT* combines transform techniques within a large multiplication algorithm [6]. In this paper, *hybrid* means CPU/GPU division of execution work. These are different concepts. NeoCruncher does not claim to implement Yee's Hybrid NTT. A shared term must not be taken as algorithmic equivalence or a claim of comparable optimization.

The reference to y-cruncher identifies technical influence, not endorsement, shared authorship, or benchmark equivalence. No controlled y-cruncher run matching the present configurations is available. Accordingly, this paper makes no quantitative speed or memory comparison with y-cruncher. Attribution also does not substitute for any license obligations attached to reused code; the release's notices remain authoritative for source attribution.

2.3 Memory reduction as a separate research problem

Reducing memory consumption can require changing what intermediate information is retained, not merely changing a displayed estimate. Mezzarobba analyzes truncated binary splitting for

D-finite function evaluation [7]. Such techniques motivate future work but are not implemented or validated by the measurements in this paper. The current engine retains exact large intermediates and is not a general disk-backed, out-of-core solver.

3 Mathematical foundations

3.1 Notation and the requested output

Let D denote the requested number of decimal places, n the number of series terms, and $M(b)$ the cost of multiplying integers of at most b bits in the selected arithmetic backend. The intended output for a positive constant x is its *truncated* expansion, corresponding to

$$z_D = \lfloor 10^D x \rfloor. \tag{1}$$

The decimal point is inserted D characters from the right after z_D is converted to base ten. Truncation is distinct from rounding to nearest. Guard digits and enclosing bounds are needed to decide which integer in Equation (1) is justified.

A useful separation is between the value being computed, the exact integer representation of a finite approximation, and the argument that the approximation determines the required digits of the infinite expression. An exact partial sum does not, on its own, establish the last step.

3.2 Chudnovsky's representation

The primary large-scale workload is pi evaluated through Chudnovsky's formula, whose form is also documented in Yee's formulas reference [5]:

$$\frac{1}{\pi} = \frac{1}{426880\sqrt{10005}} \sum_{k=0}^{\infty} \frac{(-1)^k (6k)!\,(13591409 + 545140134k)}{(3k)!\,(k!)^3\,640320^{3k}}. \tag{2}$$

Set $A = 13591409$, $B = 545140134$, and $C = 10939058860032000$ ($C = 640320^3/24$). For $k \geq 1$, define

$$p_k = (6k-5)(2k-1)(6k-1), \quad q_k = Ck^3, \quad t_k = (-1)^k p_k (A + Bk), \tag{3}$$

with the special initial leaf $(p_0, q_0, t_0) = (1, 1, A)$. The positive term ratio factors have been arranged so that products of p_k/q_k recover the factorial factors in Equation (2).

The exponential scale 640320^{3k} gives about 14 decimal digits of convergence per additional term, once polynomial factors are accounted for. This explains the implementation's initial choice $n = \max(2, \lfloor D/14 \rfloor + 4)$. That choice is a starting estimate, not a substitute for the enclosing-interval check. The code can increase the term count if the requested prefix is not resolved.

3.3 What a binary-splitting node represents

For a half-open interval $[a, b)$, define

$$R_{a,b} = \prod_{j=a}^{b-1} \frac{p_j}{q_j}, \tag{4}$$

$$S_{a,b} = \sum_{k=a}^{b-1} \frac{t_k}{q_k} \prod_{j=a}^{k-1} \frac{p_j}{q_j}. \tag{5}$$

An empty product is one. A node (P, Q, T) represents the pair of rational quantities $R = P/Q$ and $S = T/Q$. For adjacent intervals $[a, m)$ and $[m, b)$, the first interval contributes S_L directly; every term in the second is multiplied by the factor accumulated across the first. Hence

$$R = R_L R_R, \qquad S = S_L + R_L S_R. \tag{6}$$

Clearing denominators yields the implementation's recurrence:

$$P = P_L P_R, \quad Q = Q_L Q_R, \quad T = T_L Q_R + P_L T_R. \tag{7}$$

For a root beginning at zero, T/Q is the finite Chudnovsky series sum. Thus the corresponding finite approximation to pi is $426880\sqrt{10005}\,Q/T$.

Proposition 1. *The merge in Equation (7) preserves the meanings in Equation (5) and is associative for contiguous intervals with nonzero denominators.*

Proof. Substituting P_i/Q_i and T_i/Q_i into Equation (6) gives Equation (7). For three adjacent intervals the sum becomes $S_1 + R_1 S_2 + R_1 R_2 S_3$ under either parenthesization, and the product is $R_1 R_2 R_3$. Exact tuple merging also gives the same unreduced integer tuple by distributivity and associativity of integer multiplication and addition. □

For example, merging the illustrative nodes $(2, 5, 3)$ and $(7, 11, 13)$ gives $(14, 55, 59)$. Its sum ratio is $59/55 = 3/5 + (2/5)(13/11)$. These are synthetic nodes chosen to illustrate the recurrence, not Chudnovsky leaves. The example makes clear why independently computed subtrees can be combined without revisiting their individual terms.

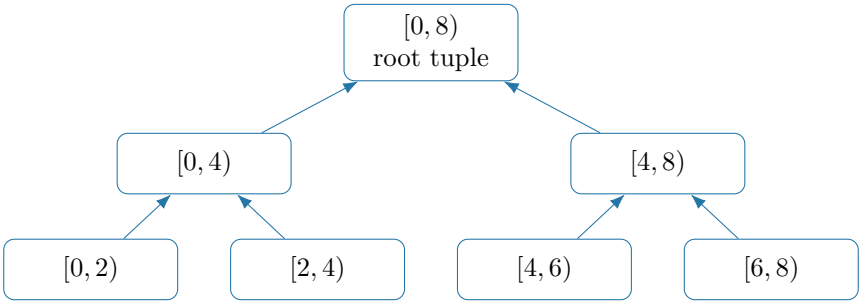


Figure 1: A binary-splitting tree. Arrows show data moving from completed subintervals toward the root. Nodes near the root operate on larger integers; equal numbers of nodes do not imply equal execution times.

3.4 Why splitting can improve computation

A naive implementation repeatedly forms and combines rational terms. It may perform expensive divisions, denominator growth, or normalization at an unhelpful granularity. Binary splitting defers the final division and exposes balanced products, which can benefit from fast integer multiplication. It also creates independent subtrees for parallel execution.

This does not mean that every merge has the same cost. At level ℓ there are fewer nodes, but their operands are larger. A useful accounting model is

$$T_{\text{tree}} \approx \sum_{\ell} N_\ell \{4T_{\text{mul}}(b_\ell) + T_{\text{add}}(b_\ell) + T_{\text{schedule},\ell}\}, \tag{8}$$

for triple nodes, omitting distinctions between unbalanced products. If the root operands have size $b_{\max}$ and the multiplication model has the usual regular growth properties, a balanced-tree

upper accounting bound is $O(M(b_{\max}) \log n)$. The exact unreduced factorial-like products may have $O(n \log n)$ bits, even when the requested output has $O(D)$ bits. One must therefore not replace $b_{\max}$ by D without analyzing the representation. This paper makes no new asymptotic complexity claim for the whole implementation.

3.5 Taylor evaluation of e and other constants

For e, the primary identity is

$$e = \sum_{k=0}^{\infty} \frac{1}{k!}, \qquad 0 < e - \sum_{k=0}^{m} \frac{1}{k!} < \frac{1}{m\, m!}, \quad m \geq 1. \tag{9}$$

To see the bound, write the remainder as $1/m!$ times a sum of products of reciprocal integers, each bounded by powers of $1/(m+1)$. The resulting geometric sum is $1/m$, with strict inequality in the full tail. A two-variable node (F, E) stores a factorial product and the numerator of its partial sum; adjacent nodes merge as

$$(F_L, E_L) \circ (F_R, E_R) = (F_L F_R,\ E_L F_R + E_R). \tag{10}$$

The global root starts at index one, and adding the $k = 0$ term gives e. Its factorial tail allows the term count to grow far more slowly than in the elementary limit $(1 + 1/n)^n$. Indeed, $n \log(1 + 1/n) = 1 - 1/(2n) + O(n^{-2})$, so that limit has error $e/(2n) + O(n^{-2})$. It is a poor basis for very high precision compared with factorial-tail series.

The application also has integer-root paths for $\sqrt{2}$ and the golden ratio, and additional series families. These share infrastructure but are not automatically qualified by pi benchmarks. User-supplied function specifications likewise require their own domain, convergence, and accuracy arguments. A successful parser or a matching small output does not establish correctness for an arbitrary custom function.

3.6 Enclosing intervals and the final decimal prefix

Let S_N and S_{N+1} be adjacent Chudnovsky partial sums. Under the standard alternating-series condition on decreasing term magnitudes, the infinite sum lies between them. Let $s_- \leq S \leq s_+$, with $s_- > 0$. The implementation uses $H = 10^{D+24}$ and the integer $r = \lfloor H\sqrt{10005} \rfloor$. Then

$$\frac{426880r}{10^{24} s_+} \leq 10^D \pi \leq \frac{426880(r+1)}{10^{24} s_-}. \tag{11}$$

All rational bounds can be evaluated with integer products and floor divisions using the node numerators and denominators. If the floors of the two bounds are the same integer, Equation (1) is determined. Otherwise, the implementation adds terms and retries within its configured attempt limit, or reports failure instead of silently presenting an unresolved prefix.

The reasoning is conditional on the series identity, tail inequalities, and correct implementation of the arithmetic and bounds. It is not a machine-checked proof of the program. Independent output comparison addresses a different source of confidence and is discussed in Section 8.

4 System architecture

4.1 Components and contracts

NeoCruncher is the application layer: it handles the terminal workflow, requests, resource preflight, progress events, backend selection, results, and metadata. Its classical numerical layer

defines series recurrences and constant-specific finalization. The CPU host tree keeps large integers in GMP's `mpz` representation when available.

Kolmogorov is the arithmetic and OpenCL integration layer. It includes a host-returning exact multiplication interface and a device-resident integer runtime. The integration API has its own version identifier, `1.2.0`; this is separate from the Alpha component identifier `1.0.0a1`. Capabilities and supported operations must be checked explicitly rather than inferred from similar version strings.

A selected *mode* describes arithmetic placement: CPU, GPU-pure, or hybrid. A selected *driver* describes a transform implementation, such as native or Kolmogorov. A selected *CPU provider* describes host subtree execution, such as the existing provider or PyKolm. These axes are related by supported combinations; they are not interchangeable labels. The scale results discussed here use the existing CPU provider, recorded as `legacy`, rather than establishing PyKolm performance.

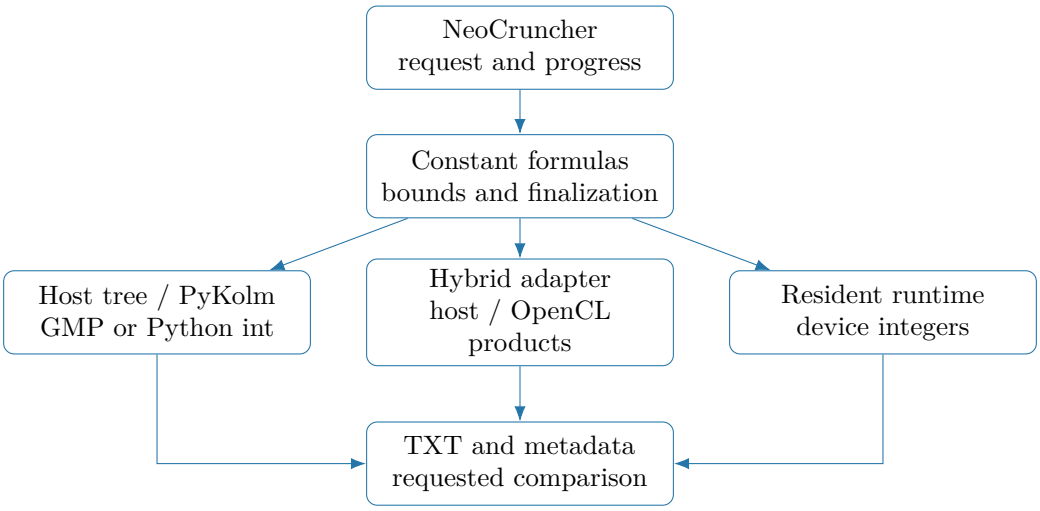


Figure 2: Logical component boundaries. PyKolm is a CPU subtree policy; the resident runtime uses OpenCL arithmetic. This diagram does not imply that all algorithms support all backends.

4.2 PyKolm: explicit CPU plans and measured subtrees

PyKolm is an experimental Python-facing CPU execution layer, identified as `0.1.0` in the inspected source. Its purpose is to make subtree plans, normalization choices, and measurements explicit. It is not a new limb arithmetic implementation, a compiler, or an OpenCL backend. Integers remain GMP `mpz` when gmpy2 is present, with Python integers as a fallback.

A `Plan` specifies a series family, interval $[a, b)$, an integer family argument, a leaf-block size, and whether reduction is enabled. The present families are Chudnovsky (`chud`) and exponential series (`exp`). The default leaf block contains 32 terms; validated plans allow sizes from 1 to 4096. Exponential plans start at index one and support the alternating flag used by the inverse-exponential route. These are concrete supported families, not a claim to execute arbitrary mathematical functions.

A `Session` executes the plan and accumulates metrics. It supports a local recursive tree and, for sufficiently large intervals and multiple workers, a bounded sequence of subprocess tasks. A thread pool coordinates the subprocesses; separate Python processes perform their arithmetic. Plans are serialized as JSON, and node values are returned as signed hexadecimal strings. This protocol avoids a dependency on decimal rendering for internal transport, but still has encoding, decoding, and process-startup costs. It does not provide zero-copy sharing of large integers.

The parent consumes adjacent subtree results in order and combines them with a height-indexed frontier. Completed nodes of equal height are merged, much like carrying into the next binary digit. This bounds the number of stored frontier tuples, but does not make their integer payload constant-size. Subprocess outputs, pending results, and serialization buffers also consume memory. The logical payload metric is therefore not a process memory peak.

Optional exact normalization. For a Chudnovsky triple, PyKolm can compute $g = \gcd(P, Q, T)$ and replace the tuple by $(P/g, Q/g, T/g)$. The current implementation attempts this only when the maximum component bit length is at most 8192. It does not normalize the exponential pair using the same rule. Both P/Q and T/Q are preserved, which are exactly the quantities needed by Equation (6).

More explicitly, merging two triples is bilinear. Dividing one input by a common factor g and the other by h divides every merged component by gh. Thus normalized and unreduced trees represent the same rational quantities, even when their integer tuples differ. Tests of a reduction policy must compare these ratios or final results, not demand componentwise equality with an unreduced tuple. This argument justifies the transformation; it does not predict its speed. Greatest-common-divisor work can cost more than the arithmetic it saves.

PyKolm records node counts, reduction counts, accumulated cancelled-factor bit lengths, GCD elapsed time, maximum node payload, sampled frontier payload, child-task counts, pending-task limits, and local split/merge timings. Cancelled factor bits are an aggregate diagnostic, not bytes of RAM saved. No supplied large-scale benchmark isolates PyKolm from the existing provider; consequently, this paper claims neither a measured speedup nor a measured memory reduction for PyKolm. It provides a controlled place to perform those experiments later.

4.3 User workflow and observable progress

The application exposes precision, backend, selected resources, and output locations. Progress events distinguish the active phase from a quantified work counter. Completed merge counts can describe a current level; emitted ASCII byte counts can describe decimal conversion. Neither is automatically a percentage of elapsed runtime, since upper merges cost more than lower ones. A long indivisible library operation can be reported as an indeterminate active phase without inventing a completion estimate.

The intended experience is a sober terminal workflow with sufficient status to distinguish computation from inactivity. The architectural requirement is that numerical stages emit progress information; this paper does not certify that every supported path has identical update frequency or responsiveness. Custom functions and long native calls need their own responsiveness tests.

5 CPU execution

5.1 Blocked host arithmetic

The default host tree evaluates small blocks before merging them recursively. For Chudnovsky, the leaf block starts with $(1, 1, 0)$ and applies the term recurrence locally. The current default is 32 terms. Small leaf work can use Python integers, after which values are converted to GMP integers and retained in that representation above the leaves. This avoids converting every large operand for every individual multiplication.

Retaining a representation changes allocation and conversion costs, not the mathematical formula. It can be valuable even if the multiplication algorithm inside GMP remains unchanged. Conversely, a fallback to Python integers is functionally useful but should not be assumed to have the same performance or memory profile as the GMP configuration.

5.2 Multiprocessing and the coordinator

For sufficiently large workloads, the default CPU path partitions the term interval into contiguous subintervals and submits them to a process pool. The implementation chooses `forkserver` when available and otherwise `spawn`. The parent consumes results in interval order, releases consumed future references, and combines nodes using a balanced frontier. This removes the requirement to retain every completed root until one final linear merge.

Ordered consumption preserves the series order, but can wait for an early subtree while later results are ready. The recorded wait time therefore includes worker arithmetic, scheduling and transport effects. It is not a measurement of idle CPU cycles. The recorded parent merge time is elapsed time inside the coordinator's combining work; it is nested within the overall binary-splitting interval.

The worker count describes requested coordinator processes. It does not imply that every stage uses that many cores or that GMP performs every individual multiplication with that many threads. Parent merging, square-root/bound work, and decimal conversion can remain substantial serial or differently parallelized portions. Tiny tasks may execute locally because startup cost would dominate.

5.3 Finalization and decimal conversion

For pi, the CPU finalizer computes the scaled integer square root and evaluates the neighboring rational bounds. Once the two floor values agree, it converts the scaled integer to decimal. With gmpy2 available, rendering uses GMP's base-ten conversion; a divide-and-conquer integer-to-decimal fallback is also provided. Formatting a decimal string is computational work, not just writing bytes to a file.

The application retains a complete output string before saving the final TXT. For pi with D places and a final newline, the TXT contains $D + 3$ bytes: `3.`, D decimal characters, and a newline. Temporary integers, multiple string representations, and file buffers can make working memory much larger than this output size. The byte count is a lower bound for output storage, not a memory requirement estimate for the entire calculation.

5.4 Memory planning versus enforcement

The planner estimates costs from the series' large intermediate products, worker count, and implementation overheads. Such estimates remain heuristic and may intentionally cover a wide range. Physical RAM, available RAM, job budget, and GPU memory are distinct quantities; showing available memory does not mean that the program failed to detect installed RAM.

Resource sampling can stop a job after an observed limit is exceeded, but it is not an operating-system reservation and can miss short peaks. Lowering an estimate does not reduce the actual number of allocated bytes. Likewise, adding swap does not turn the numerical engine into a designed out-of-core algorithm. These distinctions are essential when interpreting billion-place results or apparent allocation failures.

6 OpenCL GPU execution

6.1 Host-returning and resident interfaces

A host-returning multiplication interface accepts host integers, transfers operands to the device, executes multiplication, and returns a host result. It can be used by a hybrid constant solver,

but does not make that solver GPU-pure. In particular, CPU additions, division, or decimal conversion remain CPU arithmetic when performed by the caller.

The GPU-pure route instead uses a `DeviceInteger` abstraction. Its values are stored as signed base-2^{16} magnitudes; arithmetic operations dispatch to OpenCL kernels. The Python host coordinates indices, precision, scheduling, metadata and storage. Converting a device integer into a host integer is explicitly rejected by the object interface. This helps prevent an accidental host-arithmetic shortcut, although source-level restrictions are not independent execution attestation.

The resident runtime can inject a native or Kolmogorov transform driver. These drivers share the device-integer interface but retain distinct transform schedules. This arrangement separates numerical orchestration from the implementation of the convolution. The supported complete resident constant paths are Chudnovsky pi, Taylor e, integer-root $\sqrt{2}$, and the golden ratio. A different algorithm must be explicitly implemented or rejected rather than silently counted as a GPU-pure success.

6.2 Exact convolution, CRT, and carries

Let $A = \sum_i a_i \beta^i$ and $B = \sum_j b_j \beta^j$, where $\beta = 2^{16}$ and $0 \le a_i, b_j < \beta$. Before carrying, their product has coefficients

$$c_k = \sum_{i+j=k} a_i b_j. \tag{12}$$

A number-theoretic transform evaluates this convolution modulo a prime using roots of unity. Zero padding to a sufficient transform length prevents cyclic wraparound from changing the desired linear convolution. Inverse transforms recover coefficient residues.

The inspected resident implementations use primes 2013265921 and 1811939329. Their residues are combined through the Chinese remainder theorem. For nonnegative coefficients, unique reconstruction of an ordinary coefficient requires a bound smaller than the product of the moduli. A useful bound for untiled inputs is

$$c_k \le \min(L_A, L_B)(\beta - 1)^2, \tag{13}$$

where L_A, L_B are the operand limb lengths. Tiled multiplication must also account for exact accumulation of tile contributions. These range conditions, transform-length support, and machine-word arithmetic bounds are correctness requirements, not optional performance settings.

Recovered coefficients are not yet normalized base-β digits. Carry propagation turns them into a canonical magnitude; signs are handled separately. This is one reason raw transform throughput does not equal end-to-end arbitrary-precision multiplication throughput. Setup, root tables, packing, CRT reconstruction, carrying, allocation, and synchronization all contribute to elapsed time.

6.3 Division, roots, and decimal output on the device

A complete constant solver needs more than multiplication. The resident path implements integer division using reciprocal refinement followed by exact correction, and square-root operations using device arithmetic. For division, a candidate quotient must satisfy $a = qb + r$ with $0 \le r < b$ for positive b. Fast approximation is useful only if correction restores this exact contract.

Decimal rendering recursively splits a value by powers of ten. Equal-width subproblems reuse cached reciprocal information for the same divisor rather than rebuilding it for every node. Small output leaves are batched, and progress reports emitted characters. The final decimal bytes are downloaded for storage. This download does not imply that the high-precision numerical conversion itself was performed on the CPU.

These optimizations reduce particular repeated costs in the design. The available evidence does not establish that they resolve the earlier 10M GPU-only timeouts on the target device. That requires a new completed GPU-only run with the relevant source identity and counters.

6.4 Residency, spill, and profiling

Device buffers can be copied to RAM as storage and later restored when GPU memory is constrained. Such spill changes residency and transfer cost, not arithmetic placement: the intended numerical operations remain OpenCL kernels. It does not provide a general disk-backed numerical engine, and working transform buffers still have device allocation constraints.

OpenCL event profiling reports timestamps for device commands when profiling is enabled [8]. A sum of kernel event durations excludes host work, compilation, and uncounted transfers, and need not equal the end-to-end critical path when work overlaps. The runtime's kernel counters and host-operation counters are self-reported instrumentation. They help identify routing mistakes; they are not tamper-resistant proof of what executed.

7 Hybrid execution

7.1 Division of labor

The hybrid solver builds exact host subtrees, uses selected OpenCL products for upper-level combinations, and returns to the CPU for interval finalization and decimal rendering. It is therefore a heterogeneous schedule, not a wholly resident constant computation. Its valid results can share the same numerical contract as CPU results even though their timing profile is very different.

The Alpha schedule selects a host chunk size of approximately one eighth of the term interval, rounded up to a power of two and bounded below by 1024 terms, unless a chunk size is supplied explicitly. A one-thread producer can prepare the next host subtree while the caller combines the current one. The runtime propagates the arithmetic-policy context into this producer. This creates an opportunity for overlap, not a guarantee of simultaneous peak CPU and GPU utilization.

A `multiply_many` API groups related requests. In the inspected Kolmogorov host-returning adapter those products execute sequentially under the engine lock; the capability explicitly reports no fused GPU batch. A batch-group count must therefore not be presented as evidence of one fused kernel launch. Logical grouping, command dispatch, and device fusion are separate measurements.

7.2 When offload can help

A simple per-product model is

$$T_{\text{offload}} = T_{\text{pack}} + T_{\text{upload}} + T_{\text{device}} + T_{\text{download}} + T_{\text{unpack}} + T_{\text{checks}}. \tag{14}$$

Offload is locally beneficial only if this cost is lower than the corresponding host product cost under comparable conditions. Whole-run improvement further depends on dependencies, changed operand sizes, memory pressure, and overlap. Large transfers or many small launches can outweigh a fast convolution. Changing subtree size also changes which operations occur on each device.

The reported hybrid/native measurements are slower than the CPU measurements at every matching size in the supplied set. The gap grows with size. This is evidence against claiming

acceleration for those records, but it does not isolate the cause. A causal explanation requires matching configurations and phase measurements for host work, device work, transfers and finalization. No peak-FLOPS calculation can supply those missing measurements: these kernels perform exact integer and modular operations rather than a uniform floating-point workload.

8 Correctness, verification, and evidence

8.1 Four claims that must remain distinct

1. **Completion:** a process produced an output artifact and reported success. This is a statement about execution.
2. **Artifact integrity:** the file has the expected syntax, length, and digest. This establishes which bytes are present, not their numerical truth.
3. **Independent numerical comparison:** selected or complete digits match a separately computed value or reference. Its scope must be specified.
4. **Internal mathematical enclosure:** the implementation's bounds resolve the requested truncated prefix, conditional on their mathematical assumptions and correct arithmetic implementation.

These claims reinforce one another but are not interchangeable. A digest cannot repair an incorrect recurrence. A small independent prefix does not check distant digits. An internal bound is not independent of bugs in the code that evaluates it.

8.2 Full and prefix comparison

The current independent mpmath verifier accepts at most 100,000 places. For larger jobs, an explicitly selected prefix check compares only the first 100,000 places. The status `COMPLETED_PREFIX_CHECKED` records this narrower success. It must not be paraphrased as independent verification of the entire large output.

A complete comparison to a trusted same-size TXT is a different operation, represented by `PASS_REFERENCE` in the auditor. Complete digest equality can establish byte identity when the digests and their files are trustworthy; it does not establish the mathematical correctness of an untrusted reference. Fixed modular checks on individual products similarly detect many errors, but are not a proof of equality for arbitrary unbounded integers. The engine also exposes a full exact host multiplication comparison where requested, with the corresponding extra cost.

Verification scope is checked before expensive computation. Once a calculation has produced its numerical result, the TXT and initial metadata are saved before optional independent verification. Thus a verifier-capacity error or a failed external check need not destroy a completed calculation. The metadata must retain the resulting verification status rather than label the entire operation fully verified.

8.3 Failures and partial progress

An unresolved interval, unsupported path, timeout, or verification mismatch is a result to preserve. It should not disappear from a historical comparison because a smaller or different route passed. GPU-pure eligibility requires its own route and instrumentation; hybrid completion does not qualify it. Progress percentages concern a stated work unit or phase, never an implicit certificate of finished decimal digits.

9 Experimental methodology

9.1 Sequential auditor and run identity

The separate research/Lab auditor launches one scale case after another. Within each mode/driver route it increases precision and blocks later cases after a failure. Each case has a wall-time deadline; on expiry the auditor attempts to terminate its process group and retains available reports and progress events. A blocked case is not an executed failure and must remain labeled separately.

The default CPU sequence is 1M, 10M, 100M and 1B places. The hybrid sequence is 1M, 10M, 100M and 500M; GPU-only has a separate sequence from 1k through 10M. Precision values are actual decimal-place counts. Repetitions, maximum size, selected drivers, and resource budgets can be set explicitly. The public Vanilla archive excludes this auditor and its historical database.

Each comparison key includes the algorithm, backend, arithmetic driver, CPU provider, digits, worker count, verification scope, relevant policy settings, library versions, and hardware identity. A source fingerprint distinguishes code snapshots sharing the same version string. Hardware hashes separate otherwise incompatible histories, but do not make raw logs anonymous or complete: original reports can contain local paths and device information.

9.2 Timing and memory definitions

Process wall time includes startup, computation, output and the requested independent check. *Calculation time* is the interval reported by the numerical implementation; it is not interchangeable with process wall time. Nested phase timers must not be added to inclusive parent timers. A separate residual category records time not assigned to the named top-level phases; calling it residual does not mean it is pure overhead.

The outer sampler sums process-tree RSS at approximately 50 ms intervals. The application supervisor obtains worker-tree RSS/PSS at a nominal 0.5-second cadence with sampling work added between waits. RSS can count shared pages in multiple processes; PSS apportions shared mappings [9]. Main-process high-water RSS excludes descendants. Samples are non-atomic and can miss short peaks or exited processes. Missing or incoherent procfs data must remain unavailable rather than become a zero-memory claim.

The field `sampler_cpu_wall_cost_seconds` sums elapsed resource-read intervals measured with a wall clock. Despite its name it is not CPU time. At large sizes this measurement can itself be expensive. It may overlap calculation, and subtracting it from the total runtime does not recover an uninstrumented runtime. An instrumented/uninstrumented paired experiment would be needed to quantify the effect.

9.3 Evidence available for this paper

The main CPU evidence is an attached Alpha summary with four cases and a common source fingerprint. It reports Python 3.14.4, gmpy2 2.3.2, mpmath 1.4.1, NumPy 2.5.4 and PyOpenCL 2026.1.4, with 12 requested CPU processes. Detailed device models and identifying paths are omitted from the public review data pending the author's disclosure decision. This omission limits external reproducibility and is a publication issue to resolve, not a reason to infer equivalent performance elsewhere.

The supplied CPU source snapshot is the Lab execution snapshot; the later Vanilla archive includes packaging, attribution and presentation changes. The benchmark must therefore remain associated with its recorded fingerprint rather than be silently assigned to the exact bytes of the final archive. The 1B TXT itself was not available for full inspection. The newest hybrid

numbers are a console transcript without the corresponding complete JSON, source fingerprint, phase counters or memory samples.

10 Preliminary results

Table 1 summarizes the observed range without replacing the separate benchmark report. All CPU rows in that report use a 100,000-place independent prefix check. The hybrid console labels completion with a prefix check, but its detailed verification metadata has not yet been reviewed.

Table 1: Selected supplied Chudnovsky observations. Wall times are single reported trials; hybrid metadata is incomplete. No cross-backend speedup is claimed.

Route	Places	Wall seconds	Evidence
CPU/native	10^7	5.699	Alpha JSON summary
CPU/native	10^8	78.068	Alpha JSON summary
CPU/native	10^9	1176.480	Alpha JSON summary
Hybrid/native	10^7	37.420	Console transcript
Hybrid/native	10^8	1929.621	Console transcript

At 1B, calculation time is 1174.762 seconds and sampled process-tree RSS is 15.678 GiB. The binary-splitting phase accounts for 544.404 seconds, decimal conversion for 235.888 seconds, and the residual for 394.470 seconds. The 384.412-second parent merge timer is *inside* binary splitting; it is not an additional top-level category. The residual includes insufficiently separated work and cannot yet identify one optimization target.

The hybrid/native console gives 1.827 seconds at 1M, 37.420 seconds at 10M, and 1929.621 seconds at 100M. Relative to the CPU process wall figures at those sizes, the descriptive ratios are approximately 2.23, 6.57 and 24.72. These are ratios between reported observations, not controlled estimates of backend effects. They support describing the hybrid as functional at the reported sizes, with a serious performance limitation.

Earlier GPU-only 10M attempts using the native and Kolmogorov routes were reported to reach the 1200-second deadline during decimal conversion. They remain known unsuccessful stress observations. New hybrid completion does not supersede or resolve those GPU-only failures.

A historical 1.2 CPU 100M record reports 96.271 seconds of calculation versus 77.555 seconds in the Alpha summary. The numerical decrease is 19.44%, with matching reported output digests and 12 requested workers. Because this is not a replicated controlled comparison, it cannot identify a causal version improvement or isolate a particular implementation change.

11 Discussion and limitations

11.1 The CPU coordinator is an important measured cost

The 1B parent merge timer is about 32.72% of calculation time. Even if more workers accelerated subtree construction, the same coordinator cost could limit whole-run scaling. This observation motivates measuring the merge schedule separately, but does not prove that all of it is removable serial work. Operand allocation, native arithmetic and transport can interact with memory pressure and worker activity.

Decimal conversion consumes about 20.08% of calculation time at 1B. Optimizing series evaluation alone cannot remove it. The 33.58% residual is large enough that further attribution is necessary before proposing a confident architectural rewrite. A performance claim should

identify the timed region that changed, not simply associate a faster total with a new component name.

11.2 Why a completed hybrid is not yet an accelerator

The workload's suitability for a GPU depends on data movement and arithmetic intensity at the actual operand sizes. A nominally powerful device may be underused by a fine-grained host-returning schedule. Conversely, moving more work into host subtrees can reduce transfers while reducing GPU utilization. Both effects can occur without changing the mathematical result.

The next useful experiment is a matched comparison of subtree policies, including transfer bytes, launches, host phases and device event times. It must use equal verification settings and identify the source snapshot. The present console records do not permit a conclusion about which stage caused the 100M runtime. No architectural change is evaluated merely by writing down the proposed schedule.

11.3 Threats to validity

There is one supplied observation per size, without a reported variance, warm-up campaign, power/thermal trace, or instrumented/uninstrumented pairing. The public review data omit device models; the final archive has not been measured as a byte-identical artifact. The latest hybrid metadata and the complete 1B TXT remain unavailable for inspection. Sampling can perturb the workload and miss true peaks. Internal counters can be incorrect or tampered with. Independent numerical comparison covers only a prefix at large sizes.

The measured scale workload is Chudnovsky pi. It does not establish equivalent performance, correctness, progress responsiveness or resource behavior for all constants, custom functions, CPU providers, or drivers. In particular, the CPU results must not be credited to PyKolm without a provider-specific measurement. A security audit is outside the scope of this manuscript and has not been performed as part of writing it.

12 Conclusions and future work

NeoCruncher and Kolmogorov provide an inspectable path from rational-series recurrences to CPU and OpenCL execution, with explicit output and verification semantics. PyKolm adds an experimental CPU planning layer whose mathematical normalization can be explained independently of any performance claim. The auditor preserves route identity, incomplete runs and verification scope, which are prerequisites for useful version-to-version comparisons.

The supplied CPU report records completion through 1B places; the newer hybrid console records completion through 100M. The current evidence supports these limited execution observations and the independently checked prefix. It does not establish GPU acceleration, complete independent billion-digit validation, or a new arithmetic complexity result. The initial delivery scope remains CPU 100M and hybrid 10M.

Future numerical work should isolate CPU finalization and merging costs, compare PyKolm with the existing provider under equal settings, qualify the GPU-only decimal path at the previous failure size, and evaluate hybrid transfer policies on actual hardware. Memory-reduction techniques require explicit error and representation arguments. These are proposed experiments, not features or improvements already established by the current results.

Acknowledgements

We explicitly acknowledge **Alexander J. Yee** for his work on **y-cruncher** and for the public technical explanations that informed our understanding of high-precision computation and system design. We credit the Chudnovsky brothers for the pi formula used in the principal workload, and the authors of the established arithmetic and series-evaluation methods cited in this paper. We thank the maintainers of GMP, gmpy2, Python, NumPy, PyOpenCL, mpmath and the OpenCL ecosystem on which the software depends. These acknowledgements do not imply review or endorsement by those authors.

AI assistance disclosure. GPT / ChatGPT / Codex assisted with implementation, documentation, audit tooling, analysis and manuscript drafting. This collaboration is explicitly acknowledged. The project authors remain responsible for checking the mathematical statements, source attribution, empirical claims and final publication.

Availability and release status. The project's own current source is designated GNU GPL version 3 only, with original contributor notices, historical grants and third-party conditions retained. The Vanilla distribution and the separate research/Lab auditor have different contents. Final repository identifiers, release asset links, logos and full hardware disclosure should be inserted only after author review. This manuscript is not a release approval or a security certification.

A Executable structure of binary splitting

The following is explanatory pseudocode, not a replacement implementation. The production path groups small leaves and uses the selected integer backend.

```
function chud_node(a, b):
    if b - a == 1:
        return chud_leaf(a)
    m = floor((a + b) / 2)
    P1, Q1, T1 = chud_node(a, m)
    P2, Q2, T2 = chud_node(m, b)
    return P1*P2, Q1*Q2, T1*Q2 + P1*T2

function normalize_triple(P, Q, T):
    g = gcd(gcd(P, Q), T)
    return P/g, Q/g, T/g

function consume_adjacent_subtrees(results):
    frontier = []
    for node in results_in_interval_order:
        height = 0
        while frontier is not empty and frontier.last.height == height:
            node = merge(frontier.pop().node, node)
            height = height + 1
        frontier.push(height, node)
    node = frontier.pop().node
    while frontier is not empty:
        node = merge(frontier.pop().node, node)
    return node
```

Normalization is optional and thresholded in PyKolm. The last procedure stores nodes of differing heights and folds them in interval order at the end. Associativity makes the grouping legal; it does not make arbitrary reordering of the intervals legal.

References

[1] B. Haible and T. Papanikolaou. *Fast Multiprecision Evaluation of Series of Rational Numbers*. Author-hosted technical note. Bibliographic edition/year not asserted. https://www.ginac.de/CLN/binsplit.pdf.

[2] R. P. Brent and P. Zimmermann. *Modern Computer Arithmetic*. Cambridge University Press, 2010. Author's publication page: https://maths-people.anu.edu.au/~brent/pub/pub226.html.

[3] GNU MP Project. *GNU MP Manual: Multiplication Algorithms*. https://gmplib.org/manual/Multiplication-Algorithms.

[4] A. J. Yee. *Binary Splitting*. y-cruncher technical documentation, updated 12 October 2023. https://www.numberworld.org/y-cruncher/internals/binary-splitting.html.

[5] A. J. Yee. *Formulas and Algorithms*. y-cruncher technical documentation, updated 25 October 2023. https://www.numberworld.org/y-cruncher/internals/formulas.html.

[6] A. J. Yee. *The Hybrid NTT*. y-cruncher technical documentation, updated 19 July 2023. https://www.numberworld.org/y-cruncher/internals/multiplication/hybrid-ntt.html.

[7] M. Mezzarobba. *A Note on the Space Complexity of Fast D-Finite Function Evaluation*. Author-hosted manuscript, 2012. https://marc.mezzarobba.net/ecrits/Mezzarobba_TruncatedBinarySplitting_v1_2012.pdf.

[8] Khronos Group. *clGetEventProfilingInfo*. OpenCL API reference. https://registry.khronos.org/OpenCL/specs/unified/refpages/man/html/clGetEventProfilingInfo.html.

[9] Linux Kernel Documentation Contributors. *The /proc Filesystem*. https://docs.kernel.org/filesystems/proc.html.

All online technical references accessed 18 October 2026.